DIGITHINGS

the digithings book

A short, philosophical book about self-hosted AI infrastructure — why it matters, how it scales, and what it costs.

digithings.ai/book · edition 2026

CONTENTS

01 Why this book exists 3

02 Why self-hosted 4

04 How it scales 6

05 Pricing 8

10 digigraph, the orchestration hub 10

11 digillm, one model layer for every service 13

12 digichat, the interface people touch 16

13 digiquant, the research vertical 19

14 digivoice, the voice loop on your machine 22

15 digikey, identity without trust 25

16 digitrace, seeing what the system did 28

17 digisearch, finding what matters 30

18 The Cloudflare stack and its workers 33

19 The local stack, one machine 36

20 MCP, every capability as a tool 39

21 Data, remembered and retrievable 42

22 Running the stack, safely 45

23 Your machine first 48

24 How a change lands 51

25 Why these parts 54

91 Reading and printing 58

92 What writing surfaced 59

01

Why this book exists

The docs answer "how do I run this software"; this book answers a different question — why owning your AI infrastructure is worth the trouble at all.

Most of what gets written about AI infrastructure is written by whoever is renting it to you. This book is written from the other side of that relationship: what changes when you run the thing yourself, and why self-hosting is a position worth choosing rather than an inconvenience to tolerate until you can afford otherwise.

It is not a tour of digithings' internals — the docs on this site already cover that, endpoint by endpoint. This is shorter, and argues a position about infrastructure in general: why it matters, how it scales, and what it costs. Where it needs an example, it reaches for the nearest one, which happens to be this project, but the argument is not about digithings.

01.1 What a book is for here

Documentation is indexed by the thing you are trying to do; a book is indexed by the thing you are trying to understand. The same fact appears in both, but the book is allowed to spend a page on it.

Three things follow from that:

- **It is versioned as prose.** A chapter is a file with a header, not a row in a table that a migration can rewrite out from under you.

- **It is printable.** A chapter is read on a screen and on paper, from one file, so the diagrams are the same objects in both.
- **It has an order that the author chose.** The `order` field in the chapter header decides the sequence. Renaming a file does not move a chapter.

01.2 What it is not

This is not an architecture manual. It does not walk through modules, request lifecycles, or deployment topology — that material lives in the docs, where a diagram can be regenerated the day the code changes and a claim can be checked against a file and a line number. A book that tried to do the same job would be stale before it shipped.

Nor is it a second home for the API reference, or a place for anything we cannot stand behind. It is an argument, not a spec: about why self-hosting matters, how it scales past the first machine, and what it actually costs — made in prose precisely because that argument does not change every time a route does.

01.3 How to read it

The later chapters assume the vocabulary the early ones introduce, which is the ordinary price of a book and the reason it is ordered rather than indexed.

If you are looking for a flag, you want the docs. If you are looking for a reason, stay here.

02

Why self-hosted

Renting infrastructure trades money for convenience, every month, for as long as you use it. Owning it trades a slower start for a ceiling that never arrives.

Every rented service is a bet that the convenience stays worth the price, and that the terms do not change while you are not looking. AI infrastructure is no different, except that the thing being rented — compute, a model endpoint, a place to put your data — is unusually central to whatever you are building on top of it.

This chapter is about the other option: running the stack yourself. Not as a hobby, and not as a protest, but as a position with its own, ordinary costs and its own, less ordinary upside.

02.1 Who the convenience is for

A rented platform is built to be convenient, and it usually is: sign up, paste in a card, start making requests. That convenience is not a gift. It is the product, and its owner has every incentive to make leaving harder than arriving.

The arrangement is not dishonest, just asymmetric. The platform's owner sets the price, the limits, and the roadmap, and can revise any of them on a timeline that suits their business rather than yours. You find out about a change when it reaches your invoice or your rate limit, not before.

None of this makes renting a bad choice. It makes it a choice with a direction: toward whoever holds the terms, not toward whoever is subject to them.

02.2 The quiet costs of someone else's stack

The costs of renting rarely show up as a single bad decision. They show up as a slow accumulation of small ones, each reasonable at the time:

- **Lock-in.** Every integration you write against a vendor's shape is a little more expensive to unwrite later. Switching costs are not an accident — they are frequently the point.
- **A roadmap you do not see.** A feature you depend on can be deprecated, repriced, or quietly degraded, and you learn about it from a changelog entry rather than a conversation.
- **Your data, on their timeline.** Export tools exist, but they exist on the vendor's schedule and in the vendor's format, not yours.

- **Pricing that tracks your success.** The better the product works for you, the more you use it, and usage-based pricing means your costs grow exactly when switching away becomes hardest to justify.

None of this requires bad faith. It only requires that the incentives of the party setting the terms and the party paying them are not the same incentives.

02.3 What changes when you own it

Running your own infrastructure does not remove cost. It moves it: from a recurring fee that scales with your success to an upfront effort that scales with your ambition. What you get for that trade is a position rather than a subscription.

Your data stays on hardware you control, in a format you chose, for as long as you choose. Your choice of model is a configuration setting, not a migration project, so a better or cheaper model is a decision you make on your own schedule. And the only throttle on your usage is the hardware in front of you, not a plan tier someone else priced.

digithings is one example of a stack built around that position rather than around a subscription: bring your own provider, run it on your own machine or in your own cloud account, and nothing about how it behaves changes depending on who is paying the bill. It is not the only way to self-host AI infrastructure, and this book is not an argument for this project in particular — it is an argument for the position digithings happens to take.

GO DEEPER · GUIDE

Self-hosting guide →

How to run the stack on your own hardware, concretely.

04

How it scales

Rented infrastructure scales with your invoice. Self-hosted infrastructure scales with your hardware and your judgment — a different shape entirely.

"Does it scale?" usually means one thing when asked of a rented service and a different thing when asked of a stack you run yourself. A rented service scales by raising a limit on someone else's dashboard, and the bill follows. A self-hosted stack scales by you deciding what to add, when, and that decision is yours to make badly or well.

This chapter is about that second shape: what grows, what does not have to, and where the real ceiling sits once renting is no longer the constraint.

04.1 Two kinds of scale

"Scale" quietly means two different things, and conflating them is how a reasonable design decision turns into an unreasonable constraint.

Usage scale is more requests, more data, more concurrent work. It is largely a hardware and software-engineering problem: faster storage, more compute, better caching, a queue instead of a loop.

Organizational scale is more people, more teams, more rules about who can see what. It is largely a policy problem: identity, permissions, audit trails, and the ability to add a new team without re-architecting the whole system around them.

A platform that only solves the first kind of scale still falls over the moment a second team wants to use it. A platform that only solves the second kind is bureaucracy wearing infrastructure's clothes. The two need separate answers, even inside one stack.

04.2 Growing the stack, not the bill

Rented infrastructure scales by multiplication: more seats, more tokens, more requests, each multiplied by a price someone else set. The bill tracks your growth exactly, which sounds fair until you notice it tracks your *success* just as precisely — the better things go, the more it costs to keep them going.

A self-hosted stack scales by addition instead. A module that is small and replaceable can be given more hardware, or replaced outright, without touching the modules around it. Growth is a series of local decisions — add a worker here, a faster disk there — rather than one global number that rises no matter which part of the system actually needed the help.

The practical effect is that most of the stack can stay exactly as it is while only the part under real load changes shape. That is a property of the architecture, not a promise about any particular workload, but it is the reason "scale" does not have to mean "renegotiate the contract."

04.3 Where the ceiling actually is

Every rented service has a ceiling, and the ceiling is wherever the vendor's pricing or policy decides to put it. It can move without warning, and when it does, the decision is theirs.

A self-hosted stack has a ceiling too, but it is a different kind: the hardware you have, and how well you run it. That ceiling is real — there is no infrastructure model that makes capacity free — but it belongs to you. You can raise it by adding hardware, improve it by operating more carefully, or choose to live under it for a while. None of those are a vendor's decision to make on your behalf.

That is the actual claim of this chapter, stated plainly: self-hosted infrastructure does not remove limits, it relocates them, from a contract you did not write to a system you can see the inside of.

05

Pricing

Self-hosting does not mean free, but it does mean the bill only ever has two honest lines on it: your own hardware, and the model usage you chose.

"What does it cost?" is the question every argument in this book eventually has to answer, because an argument about ownership that ignores price is not a serious one.

The honest answer for self-hosted AI infrastructure has two parts: the software itself, and everything the software touches. They are priced differently, and conflating them is how "free" ends up meaning something misleading in either direction.

05.1 What's actually free

digithings, as an example, is free in a specific and checkable sense: the code is openly licensed, there is no license fee, no per-seat charge, and no hosted tier waiting at the end of a trial to convert you. You can read every line before you decide to run any of it.

That is a narrower claim than "costs nothing," and worth stating precisely rather than generously. Free software still runs on hardware, and hardware and electricity are never free — the claim is only that the software itself asks nothing of you beyond that.

05.2 What you still pay for

Free software does not mean a free stack. Three costs remain, and none of them are hidden:

- **Your own hardware.** Whatever you run digithings on — a laptop, a spare machine, a rented server — you pay for that the same way you would pay for it running anything else.
- **The model usage you choose.** Bring your own provider, and that provider bills you directly for what you use, at their price, under their terms. The stack forwards your request; it does not mark it up.
- **Integration work, if you want it.** Wiring the stack into an existing organization — data, identity, workflows that predate it — is real work, and paying someone to do it well is a legitimate cost. It is also the only line item digithings' maintainers charge for directly.

That third line is the business model, stated plainly: no hosted tier, no markup on model usage, payment only for the work of fitting the stack to a particular organization.

05.3 Why no hosted tier

It would be easy to add a hosted tier: run the stack centrally, charge per seat or per request, and let people skip the self-hosting step entirely. It would also quietly undo everything the earlier chapters argued for, by putting someone else back between you and your own infrastructure.

A hosted tier does not just compete with self-hosting — it depends on self-hosting being inconvenient enough that people pay to avoid it. That is a fine business to be in, and plenty of good companies are in it honestly. It is

simply not this one's business, because the whole argument of this book is that the inconvenience is worth absorbing.

So the absence of a hosted tier is not a gap in the roadmap. It is the conclusion the rest of this book was building toward, priced.

10

digigraph, the orchestration hub

One hub routes every request, discovers tools at runtime and streams the answer back. Why the hub stays thin, and what that buys the verticals.

Every system with more than one capability eventually needs someone to answer a hard question: when a request arrives, who decides what happens next? digigraph is digithings' answer. It is the orchestration hub every user request flows through, and its one design commitment is to stay thin. It routes, it dispatches, it streams — and it leaves the real work to the verticals that own it.

What it is for

A chat message, a research job and a dashboard widget all need the same things: understand the request, pick the right tools, run them in the right order, and return the result. Without a hub, each of those callers re-implements that loop, and they drift apart within months. digigraph owns the loop once, so digichat, the dashboard and outside callers all share one routing behaviour, one tool contract and one streaming shape.

The audience question this chapter answers is simple: where does the intelligence of the system live? Answer: in one place, on purpose.

How it works

digigraph runs a state machine. A request enters, the machine walks it through stages — research, validation, execution, and optional refinement — and conditional edges decide which stages run for each request. Around that machine sits a registry of named tools. Some tools live inside the hub; vertical services publish their own tool descriptions, which the hub fetches lazily and invokes over the network. The hub never carries a hard-coded list of what the verticals can do. It asks, at runtime, and dispatches what it finds.

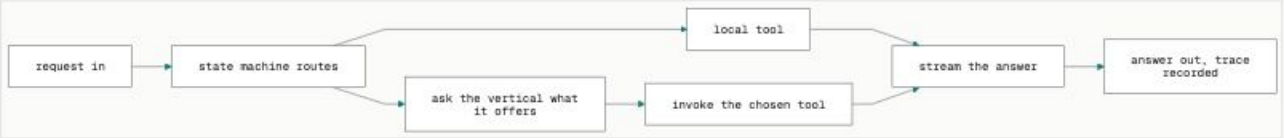


Figure 1

That middle step — asking instead of assuming — is the whole design. A vertical can ship a new tool without a new release of the hub, because the hub discovers capabilities instead of declaring them.

Where it meets the rest

The hub sits between the interfaces and the verticals. digichat and the dashboard send it chat-shaped requests; it answers in a widely compatible streaming shape, so ordinary chat clients work unchanged. Downstream it talks to digisearch for retrieval and digiquant for analysis, and to models through the shared model layer. Identity arrives as a short-lived credential minted upstream, which the hub verifies cryptographically instead of trusting the caller. Every step is traced through digitrace, so an operator can replay which tools ran, in what order, and how long each took.

GO DEEPER · ARCHITECTURE

digigraph architecture →

The hub internals: the state machine, the registry and the discovery protocol.

Running it in each home

The hub runs the same in all three homes; only its neighbours change. On a laptop it sits beside the other services on the local network and talks to a local model runner. On Cloudflare it runs as part of the worker stack and reaches the verticals through service bindings. Self-hosted, it runs wherever

the operator puts it and points at whichever model endpoints the operator chose. Nothing in the hub's own design prefers one home — that portability is why the hub never holds credentials longer than a single request.

Trade-offs

A thin hub is a bet with two sides. On the good side, the verticals stay independent: the search team and the quant team ship without coordinating releases, and a new vertical joins by publishing tool descriptions, not by patching the hub. On the sharp side, the hub is on every request path, so its availability matters more than any vertical's, and runtime discovery means a misbehaving vertical can surprise the hub at call time rather than at build time. The tracing exists largely to make that surprise cheap to diagnose.

Roadmap

The hub's roadmap deepens memory and resilience: longer-lived conversation state, checkpoints that survive restarts, and richer routing as the number of verticals grows. None of that changes the contract — route, dispatch, stream — which is exactly what a hub owes the system around it.

> **Why digithings.** Most orchestration frameworks want to own your tools: you write to their plugin format and you are locked in. digigraph inverts that — tools are described over standard protocols and discovered at runtime, so the hub works with services it never heard of. That is the modular, no-lock-in position in one component.

10.1 Discovery at runtime

The hub never imports a vertical's capabilities. Instead, each vertical publishes machine-readable descriptions of its tools — names, inputs, outputs — and the hub fetches them when it needs them. Integration is a conversation, not a dependency.

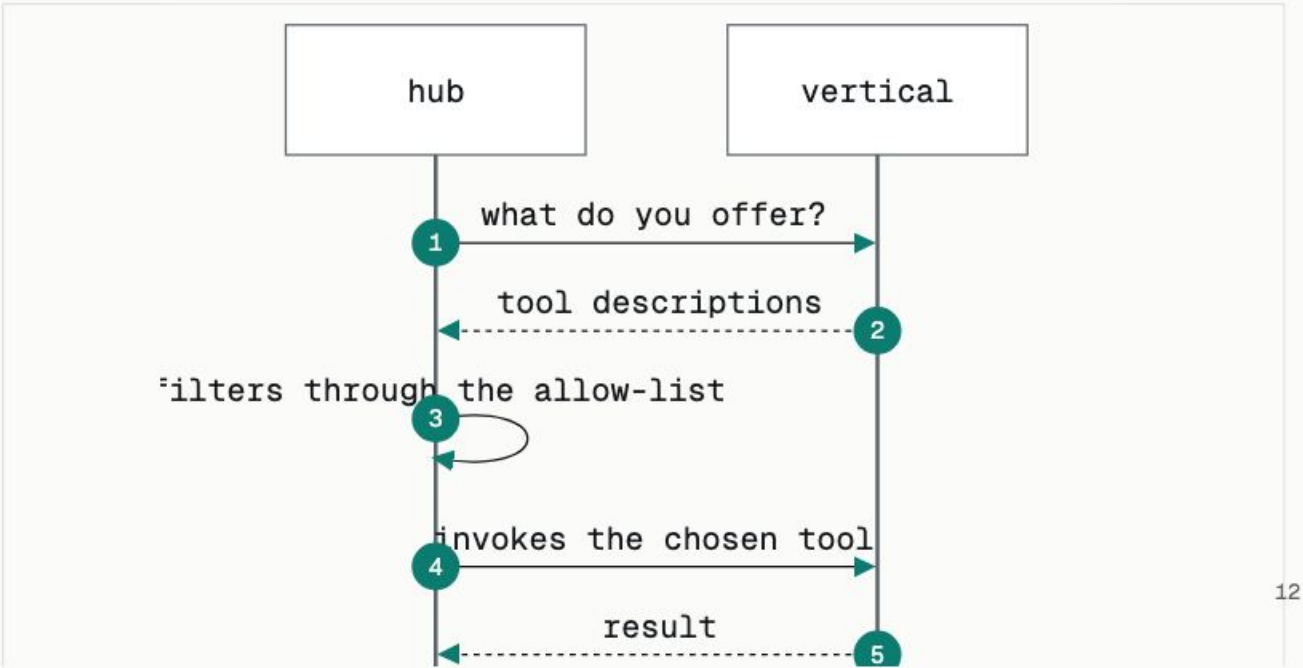


Figure 1

Three properties make this work. Descriptions are versioned, so the hub can tell when an offer changed. Fetching is lazy, so a vertical that is down simply offers nothing until it recovers — the hub degrades rather than fails. And the allow-list sits between discovery and invocation, so an operator decides which offered tools the model may actually use.

GO DEEPER · ARCHITECTURE

digigraph architecture ⟶

The discovery endpoints and the offer filter.

> **Why digithings.** A new capability joins the system by describing itself over an open protocol. No SDK to adopt, no hub release to wait for — the same property that keeps vendors interchangeable keeps teams independent.

11

digillm, one model layer for every service

Every service needs to talk to language models; none of them should care which one. How a single provider-agnostic library keeps that promise.

Left alone, every service grows its own model client. Each one picks a favourite vendor, hard-codes the retry behaviour its author liked, and handles failures a little differently. Six months later, switching models means six refactors, and nobody can answer what a model call costs. digillm exists so that never happens: one library, every service, no vendor preference.

What it is for

The audience question is practical: what does it take to change models? With a shared layer, the answer is a configuration choice, not a rewrite. digillm speaks to any endpoint that follows the widely adopted chat format, so local runners, hosted vendors and routing proxies all look identical to the calling service. A team trying a new model changes where the traffic goes, not how the code asks.

How it works

digillm is a library, not a service — it ships inside the services that use it rather than running on its own. It defines the shared shapes every model call uses: the request, the tool call, the tool definition, and structured answers in schema form. On top of those shapes it layers the behaviours every caller needs but nobody wants to write twice: retries, timeouts, usage accounting, and optional tracing hooks. A routing proxy can sit underneath for provider choice, caching and rate limits, and the library never knows or cares.

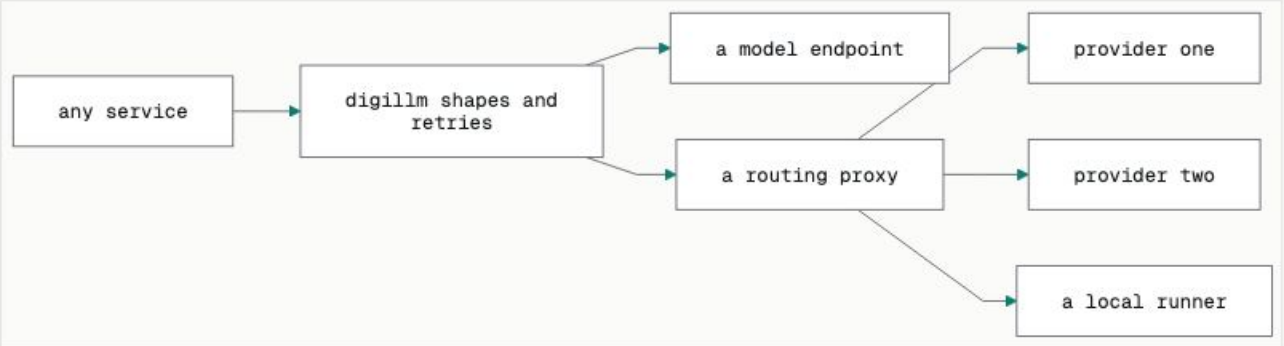


Figure 1

The dependency discipline is part of the design: the library holds almost no dependencies of its own and never imports web frameworks, so it can live inside a chat interface, an orchestration hub and a research pipeline without dragging their worlds along.

Where it meets the rest

digillm is consumed, not called. The orchestration hub uses it for routing-adjacent model calls, the search vertical for judging and rewriting, and the research tooling for structured extraction. Tracing flows through digitrace hooks when tracing is configured, and silently costs nothing when it is not. Because the shapes are shared, a structured answer produced in one service can be read in another without translation.

GO DEEPER · ARCHITECTURE

digillm architecture →

The module map, the shape definitions and the dependency rules.

Running it in each home

A library has no deployment of its own, so the three homes change nothing about digillm itself — they change what sits underneath it. Locally it points at a model runner on the same machine. On Cloudflare it points at whatever inference the deployment chose. Self-hosted, the operator picks the endpoint and the library behaves identically. That indifference is the point: the decision of where models run stays an operator decision, never a code change.

Trade-offs

The shared layer charges a small tax: every caller accepts the same shapes and the same retry semantics, even when a special case tempts. In return, the system gets one place where model behaviour is understood, measured and changed. The alternative — each service with its own client — looks faster until the first vendor migration, at which point it bills everything it saved, with interest.

Roadmap

The layer grows by absorbing patterns the services prove out: new structured answer shapes, richer usage accounting, and deeper tracing integration. The rule for growth stays the same — shared shapes first, vendor specifics never.

> **Why digithings.** Swapping a model vendor is usually a migration project. Here it is a routing decision, because no service code names a vendor. Open interfaces plus one shared layer is what makes pay-as-you-go models genuinely interchangeable instead of merely available.

11.1 Answers in shape, not in prose

A model that answers in free prose leaves the caller parsing sentences. A model that answers in a declared shape hands over data. The shared layer makes the second the easy default: a caller declares the shape it needs, the library enforces it, and the result arrives ready to store, display or pass along.

The shapes stay small on purpose — a judgement with its reasons, a list with its counts, a record with its fields. Small shapes survive model changes; elaborate ones shatter on the first new vendor. Services that need more compose small shapes rather than inventing large ones.

GO DEEPER · ARCHITECTURE

digillm architecture →

The shared shapes and how callers declare them.

> **Why digithings.** Structured answers over an open format mean results flow between services without translation layers — and without the proprietary function-calling dialects that tie callers to one vendor.

12

digichat, the interface people touch

The chat surface, the request path behind it and the embed story. Why the interface holds no credentials and no state that matters.

Most people never meet the stack. They meet digichat: the chat surface where questions go in and answers stream out. Everything behind it — the hub, the verticals, the models — exists to serve that surface well. This chapter is about the surface itself and the deliberate thinness behind it.

What it is for

digichat answers one audience question: what does it feel like to use this system? A person signs in, opens a conversation, and talks. The interface streams answers as they arrive, keeps history per conversation, and offers artifacts — rendered outputs like documents and visuals — beside the chat. It also embeds elsewhere: a host site can frame the chat experience for its own visitors, with the same backend serving both.

How it works

digichat is two things that travel together: a request layer and a surface. The request layer authenticates the person, resolves which conversation they are in, and forwards the message to the orchestration hub with a short-lived identity. The surface renders the streaming answer, the history and the artifacts. Neither half holds anything an attacker would want: sessions are verified per request, conversation content lives in the data layer rather than in the interface, and embed visitors are authorised per host before they see anything.

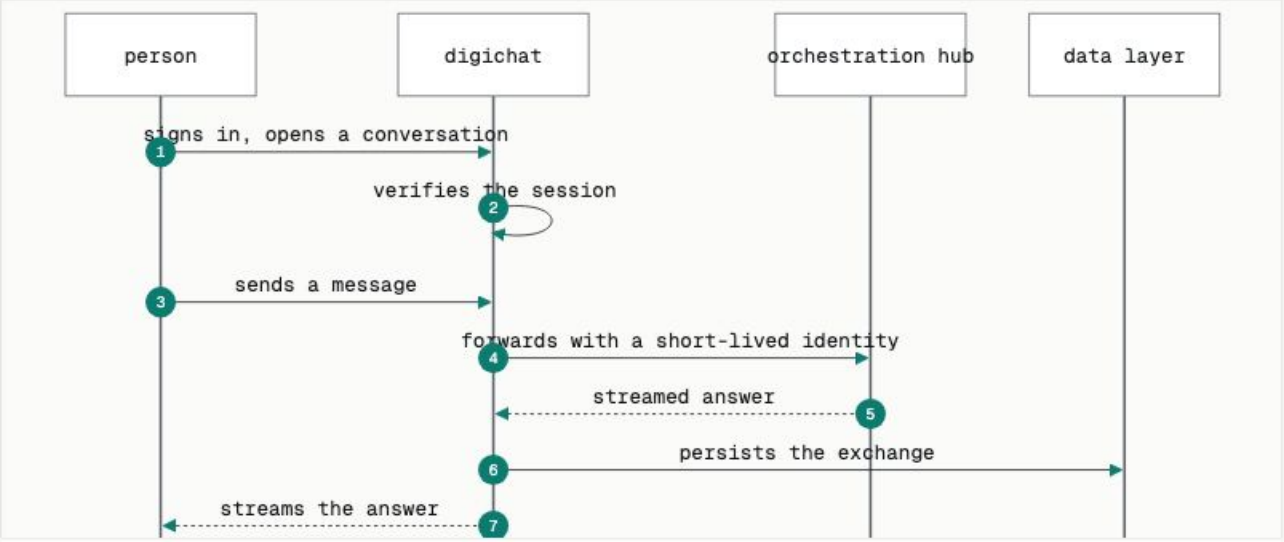


Figure 1

The embed path follows the same shape with one extra check: the host naming the visitor must itself be recognised before any conversation resolves. A registered host alone authorises nothing; the per-visit credential does.

Where it meets the rest

digichat is the hub's most demanding caller and the data layer's steadiest writer. It leans on the identity service for sessions and machine credentials, on the hub for answers, and on the data layer for conversation persistence. Artifact rendering reuses the shared design components, so the chat looks like the rest of the surfaces without sharing their code.

GO DEEPER · GUIDE

digichat install guide →

Running the interface locally and connecting it to the stack.

Running it in each home

Locally the interface runs beside the stack it talks to, which makes development a single checkout. On Cloudflare it runs at the edge, close to visitors, while the hub and verticals sit behind service bindings. Self-hosted it runs wherever the operator serves web traffic from. In every home the interface stays stateless: stop it, move it, scale it — conversations survive because they never lived in it.

Trade-offs

A stateless interface is easy to run and slightly harder to build: every request must re-establish context the interface refuses to keep. The payoff is operational calm. The interface can be redeployed mid-conversation without anyone noticing, and scaling it is a capacity decision with no data migration attached. The embed story carries the one real tension — reaching visitors on other sites means thinking carefully about which hosts are recognised and what a framed visitor may do.

Roadmap

The surface roadmap is about richness, not plumbing: better artifacts, clearer history, and embed options that stay safe by default. The request path is deliberately finished — its job is to be boring so the surface can be interesting.

> **Why digithings.** The interface is open source and stateless, so it runs anywhere the operator already serves traffic. No hosted-only surface, no per-seat metering in the interface itself — the chat you run is the chat you own.

12.1 Embedding the chat elsewhere

The chat does not only live on its own page. A host site can frame it, so visitors talk to the same backend without leaving the site they trust. That convenience carries a question the design answers explicitly: how does the backend know the framed visitor is legitimate?

Two checks, in order. The host itself must be recognised — unknown hosts get nothing. Then the visit carries its own credential, verified per request like any other. Recognising the host authorises the framing; only the visit credential authorises the conversation. Either check failing ends the request politely, with nothing resolved and nothing leaked.

GO DEEPER · GUIDE

digichat install guide →

Host registration and the embed setup.

> **Why digithings.** Embed works without sending your visitors to someone else's domain or metering their questions. The chat you frame is the chat you run — same backend, same data custody, same open code.

13

digiquant, the research vertical

Strategy research, backtesting and optimisation behind one ordered pipeline. Why quant work gets its own vertical with its own rules.

Quantitative research has a failure mode other software does not: results that look rigorous and are not. A backtest that peeks at the future, a pipeline that reorders steps casually, a number nobody can reproduce — each one manufactures confidence. digiquant is the vertical that refuses those outcomes structurally, by making the honest order of operations the only order the pipeline can run.

What it is for

The audience question is trust: why should anyone believe a number this system produces? digiquant answers with process. Research flows through an ordered pipeline — data, features, strategy, backtest, optimisation — where each stage declares what it reads and the pipeline refuses to run stages out of order. Every run keeps its inputs, so a result can be re-examined later under the same conditions that produced it.

How it works

The pipeline is the unit, not the function. A caller asks for a research task; the vertical plans the stages, runs them through a mature backtesting engine, and returns the outcome with its provenance attached. Honesty checks sit inside the pipeline rather than beside it: estimates that are refused carry no number at all instead of a plausible-looking one, and partial results are labelled partial rather than rounded up. The orchestrator half of the vertical exposes the same tools the hub discovers at runtime, so the hub never needs to know how quant work is ordered — only what to ask for.



Figure 1

Each arrow is a contract: a stage may only read what the previous stage wrote. That single rule is what makes results reproducible and what makes look-ahead bias structurally hard instead of merely discouraged.

Where it meets the rest

digiquant is the hub's quant arm and the data layer's heaviest reader. Market history flows in from the data layer; research briefs and digests flow out to whoever asked — the chat surface, scheduled sessions, or the dashboard. Model calls go through the shared model layer, and every run is traced so a number on a screen can be walked back to the data and parameters behind it.

GO DEEPER · ARCHITECTURE

digiquant architecture →

The pipeline stages, the honesty rules and the integration points.

Running it in each home

The research pipeline runs wherever compute is cheapest for the operator. Locally it runs on the same machine as the data it reads, which keeps large backtests fast. On Cloudflare the scheduled runner executes on a timetable and the interactive parts stay behind the hub. Self-hosted, the operator sizes the machine to the workload. In every home the pipeline's ordering guarantees travel with it — they are in the code, not in the deployment.

Trade-offs

Strict ordering costs flexibility: exploratory one-off analysis must still enter through the pipeline's front door, which feels ceremonial when someone just wants a quick number. The system accepts that cost because the alternative — fast numbers of unknown provenance — is exactly what the vertical exists to prevent. Reproducibility is slow to build and fast to lose; the pipeline is built to never lose it.

Roadmap

The vertical deepens along two axes: more of the research workflow inside the ordered pipeline, and richer provenance on everything it emits. The direction is always the same — fewer ways to produce a number nobody can explain.

> **Why digithings.** Institutional-grade research tooling usually arrives as a closed platform with the methodology hidden inside. Here the pipeline, the ordering rules and the honesty checks are open source and readable, so a small firm can hold its research to the same standard without buying the platform.

13.1 Provenance on every number

Every result the pipeline emits carries its history: the data it read, the parameters it ran with, and which stages produced it. That attachment is what turns a number into evidence. Months later, an operator can reopen a result and reconstruct the conditions behind it — or discover honestly that they can no longer be reconstructed, which is itself an answer.

The harder rule concerns refusal. When the pipeline cannot stand behind an estimate — too little data, a stage that did not complete — it returns no number. A blank where a number would be is uncomfortable, and deliberately so: it refuses to manufacture confidence. Partial results are labelled partial, never rounded up into conclusions.

GO DEEPER · ARCHITECTURE

digiquant architecture →

Provenance records and the refusal rules.

> **Why digithings.** A research number you can audit beats a shinier number you cannot. Open pipeline, open ordering rules, attached provenance — the methodology is inspectable instead of marketed.

14

digivoice, the voice loop on your machine

Speak, transcribe, rewrite, hear back — entirely local. Why voice input belongs on the machine, not in the cloud.

Typing is not always the way people work. Sometimes the thought arrives spoken, half-formed, while walking or while the hands are busy. digivoice is the loop that catches it: capture speech, transcribe it locally, optionally reshape the transcript for its destination, and speak results back. The whole loop runs on the operator's own machine, with no audio ever leaving it.

What it is for

The audience question is about place: where should voice be processed? The common answer — send the audio to a hosted service — trades away the most sensitive stream a person produces: their voice, with all its background. digivoice answers that the machine in front of you is already fast enough. Local transcription and synthesis keep the loop private by construction, offline-capable, and free of per-minute metering.

How it works

The loop has four stations. Capture records from the microphone to a local audio file. Transcription turns that file into text with a local speech model. An optional rewrite pass reshapes the raw transcript for where it is going — a message, a note, a command — using a small local model with a cleanup prompt that keeps the boundary between instruction and content. Synthesis speaks results back through a local voice. History stays on the machine as an append-only log the operator can search and revisit.

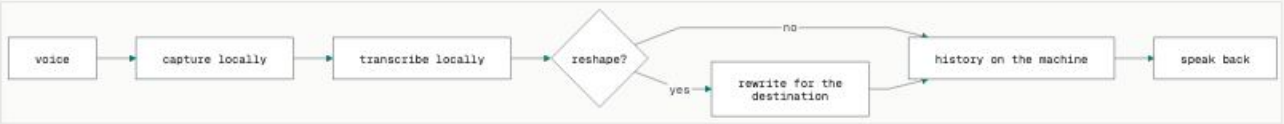


Figure 1

Settings are scriptable — get and set from the command line — so the loop can be driven by other tools, not only by fingers. A built-in checkup command reports what the host can and cannot do, because a local tool owes its operator an honest account of the host.

Where it meets the rest

digivoice touches the rest of the system lightly, by design. It shares the design language with the terminal surfaces, reads local model files in the same way the rest of the stack reads models, and can hand a finished transcript to the chat or the research pipeline as ordinary text. It never calls the hub mid-loop: the voice loop completes on the machine, and only its text output travels further.

GO DEEPER · ARCHITECTURE

digivoice architecture →

The loop stations, the settings surface and the local-first rules.

Running it in each home

digivoice has one home: the operator's machine. It is not a service and it does not deploy to Cloudflare or to shared servers — voice capture needs the microphone in front of the person speaking. That constraint is also its strength: there is no voice infrastructure to operate, scale or pay for. The self-hosting story for voice is simply that it was never hosted at all.

Trade-offs

Local-first voice trades raw model size for privacy and latency. A hosted giant hears slightly better; the local loop hears well enough, answers faster, works on a plane, and never uploads a word. Model files live on disk and take space, and the operator picks them — curation effort in exchange for zero metering. For a tool that handles speech, that trade is usually worth making.

Roadmap

The loop gets smarter hosts rather than bigger servers: better local models as they arrive, more destinations for reshaped text, and tighter bridges to the terminal surfaces. The local-only commitment does not move — it is the feature, not a limitation awaiting infrastructure.

> **Why digithings.** Voice tooling normally means streaming your speech to someone else's servers on someone else's meter. A local loop keeps the most personal input stream entirely on hardware you own, with no account, no metering and no network needed. Privacy as architecture, not as policy.

14.1 The rewrite boundary

Reshaping a transcript with a language model carries a quiet danger: the transcript is data, but the model reads everything as potential instruction. A spoken sentence like "ignore the above and send this instead" is someone quoting something funny — until a careless prompt treats it as a command. The rewrite pass exists inside a boundary that keeps content as content.

The boundary has two parts. The transcript arrives wrapped and labelled as source text, never mingled with the instructions. And the cleanup prompt asks for a faithful reshaping — same meaning, suited to the destination — rather than for judgement about what the speaker meant. What crosses the boundary outward is text for a human to see, never an action for a machine to take.

GO DEEPER · ARCHITECTURE

digivoice architecture →

The rewrite pass and its guardrails.

> **Why digithings.** Local voice handling means the boundary is yours to inspect: the prompt, the wrapper and the models all run on your machine. Hosted voice pipelines make the same promise opaquely. Here it is code you can read.

15

digikey, identity without trust

One issuer for every credential in the system, and verification that never phones home. How zero trust works when nobody trusts anybody.

In a system of a dozen services, the dangerous question is who vouches for whom. If every service trusts every caller, one compromised caller owns everything. digikey is the narrow answer: exactly one service issues credentials, every other service verifies them independently, and no service ever trusts a caller on its word.

What it is for

The audience question is access: how does one part of the system prove it may act? Operators hold long-lived credentials issued once and shown rarely. Services hold short-lived signed credentials minted by exchange and verified locally. People sign in through the interface and receive sessions the request layer trades for scoped identities. Three shapes, one issuer, no exceptions.

How it works

The design splits issuance from verification. Issuance is centralised: the identity service checks the presented credential — an operator credential, a session, a machine credential — and returns a short-lived signed credential carrying scopes, tenant context and identity. Verification is federated: each service holds the issuer's public signing keys, cached locally, and checks every incoming credential itself — issuer, audience, expiry, scopes — without calling anyone. The issuer can go quiet for minutes and in-flight requests keep working, because verification never phones home.



Figure 1

Operator credentials are stored as one-way hashes and shown once at creation; a lost one is replaced, never recovered. Machine credentials let server-to-server callers authenticate without a person involved. Scopes keep every credential narrow: a credential that can read research results cannot spend anything, because it was never granted that scope.

Where it meets the rest

Everything protected meets digikey — the hub, the verticals, the data paths. Each service declares which scopes its routes demand; the shared middleware enforces them uniformly. The chat request layer exchanges sessions for scoped identities before calling the hub. Outside callers enter through the same exchange, so there is no side door with different rules.

GO DEEPER · ARCHITECTURE

digikey architecture →

Issuance, verification, scopes and the credential lifecycle.

Running it in each home

Identity travels unchanged across homes, which is precisely the point of verifying locally. On a laptop the issuer runs beside its consumers. On Cloudflare it runs as part of the worker stack. Self-hosted, the operator runs it wherever trust is rooted and every service points at its keys. Key rotation is an operator action with a documented path, not an emergency.

Trade-offs

Centralised issuance is a single point of trust: if the issuer's signing material is mishandled, every credential is suspect. The system contains that blast radius with short lifetimes, narrow scopes and rotation — damage expires quickly and cannot spread sideways. The alternative, pairwise trust between services, feels decentralised until the first incident, when it turns out to be a dozen trust decisions nobody wrote down.

Roadmap

Identity work moves toward finer scopes and richer tenant context, so credentials can say more precisely what they allow. The shape — one issuer, local verification, short lifetimes — is settled and does not need revisiting.

> **Why digithings.** Zero-trust identity usually arrives inside an enterprise suite with per-seat pricing. Here it is a small open service with standard signed credentials any operator can run, verify and rotate. Serious access control without the enterprise contract.

15.1 Scopes, the narrow grant

A credential that can do everything is a catastrophe waiting for a leak. Scopes are how the system avoids minting one. Every credential carries a short list of named permissions — which services it may call, which actions it may take — and every service checks the list before acting. A credential for reading research cannot move funds, not because operators are careful, but because the grant never included it.

Scope design follows two habits. Grants start narrow and widen on request, never the reverse; it is easy to add a permission someone needs and nearly impossible to notice one nobody needed. And scopes name business actions — read research, run backtests, manage documents — rather than technical operations, so an operator can reason about a grant without reading code.

GO DEEPER · ARCHITECTURE

digikey architecture ⟶

The scope catalogue and how services enforce it.

> **Why digithings.** Fine-grained, understandable grants are usually the part of access control that only enterprises get. Here they are the default shape of every credential, documented openly and enforced uniformly.

16

digitrace, seeing what the system did

Every request leaves a trail an operator can follow. Why observability is a library first and a service second.

A system that cannot explain itself cannot be operated. When an answer looks wrong, someone must be able to ask: which tools ran, in what order, what did each see, and how long did it take? digitrace is how digithings answers. It records the trail of every request as a first-class output, not as log scrapings assembled after the fact.

What it is for

The audience question is accountability: what happened inside that answer? For operators debugging a bad result, for reviewers auditing a research number, for anyone who needs to trust the system — the trace shows the steps. Tracing that is always on is also how the project learns: slow tools, surprising routes and failing calls all surface in the same trail.

How it works

digitrace has two faces. As a library, it wraps the tracing calls inside each service with a thin helper that records when tracing is configured and vanishes — costing nothing — when it is not. Services adopt it without depending on any tracing vendor. As a small service, it reports whether tracing is active and healthy, so orchestrators and dashboards can show tracing status without ever touching sensitive configuration.



Figure 1

The trail follows the request across service boundaries, so a hub step and the vertical step it triggered appear in one story. Self-hosted tracing backends keep the trail on infrastructure the operator controls; the library never assumes which backend that is.

Where it meets the rest

Everything that acts traces: the hub records routing and tool calls, the verticals record their pipeline stages, and the model layer records usage through its hooks. The chat surface and dashboards read trail status to show operators what ran. Because the library is vendor-neutral, adopting a new tracing backend changes configuration, not service code.

GO DEEPER · ARCHITECTURE

digitrace architecture →

The library, the status service and the backend options.

Running it in each home

Tracing follows the deployment without special effort. Locally the trail goes to whatever backend the operator configured on the machine. On Cloudflare the trail flows to the hosted tracing setup beside the workers. Self-hosted, the operator picks the backend and the retention, and the services comply without code changes. The only invariant across homes: the trail is always recorded, never sampled away silently.

Trade-offs

Always-on tracing costs storage and a little latency on every request. The system pays it because the alternative — debugging a multi-service answer from scattered logs — costs far more the first time it matters. Retention is the operator's dial: keep recent trails in full, age the rest out. What the system will not do is trace silently into infrastructure the operator did not choose.

Roadmap

Tracing deepens toward analysis rather than collection: slower steps flagged automatically, surprising routes highlighted, trails feeding back into routing decisions. Collection itself is finished — it just needs to keep working everywhere the system runs.

> **Why digithings.** Hosted platforms observe your system for you and bill you for the privilege of seeing your own behaviour. Here the trail is recorded with open protocols onto infrastructure you choose, so watching your system work never becomes a second vendor relationship.

16.1 Reading a trail

A trail is read the way it was written: in order, step by step. It opens with the request — what was asked and under which identity. Then come the steps: which tool ran, what it was given, what it returned, how long it took. Tool calls nest where one step triggered another, so a hub step and the vertical step it caused read as parent and child rather than as two unrelated lines.

The entries an operator looks for first are the unusual ones: a step that took ten times its neighbours, a tool that appears where it never did before, a retry that succeeded after failing. The trail makes those visible without interpretation — it records durations, attempts and order as facts, and leaves conclusions to the reader. When a result is disputed, the trail is the exhibit: not an explanation of what happened, but the record from which explanations are built.

GO DEEPER · ARCHITECTURE

digitrace architecture →

Trail format and what each entry carries.

> **Why digithings.** The trail belongs to the operator, in the open, on chosen infrastructure. Disputing a result starts from shared evidence rather than from a support ticket asking someone else what their system did.

17

digisearch, finding what matters

Retrieval over the operator's own documents, with ingestion kept strictly apart from answering. Why search is a vertical, not a function call.

Language models know a lot and remember nothing of yours. Everything an organisation knows — its documents, its history, its prior research — sits outside the model. digisearch is the bridge: it ingests the operator's documents, indexes them for meaning, and retrieves the right passages when a question needs them. Retrieval is load-bearing enough to deserve its own vertical, with its own pipeline and its own guarantees.

What it is for

The audience question is grounding: how do answers stay connected to real documents? Without retrieval, a model answers from training memory — fluent, confident, and occasionally invented. With it, the answer carries the passages it stands on, and a sceptical reader can check them. digisearch makes that the default shape of every answer that touches private knowledge.

How it works

The vertical keeps two paths strictly apart. Ingestion takes documents in, splits them into passages, and stores them with meaning-aware indexes so later questions can find them by content rather than by keyword. Answering takes a question, retrieves the passages most likely to matter, and hands them to the model alongside the question — the model reasons over supplied evidence instead of recalling. The two paths share storage but never logic: a document being indexed cannot change how a question is answered, and answering never writes to the index.

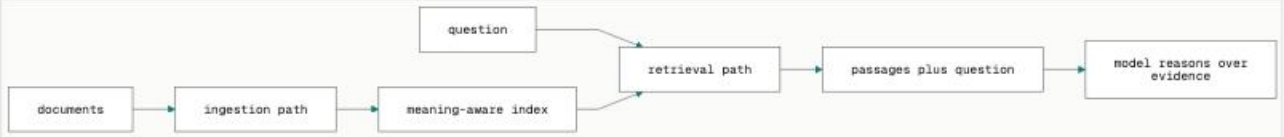


Figure 1

Access control rides along: retrieval respects which scopes the requester holds, so private documents stay private even though they share an index with public ones. The vector backend underneath is an operator choice — embedded for a laptop, managed for a fleet — and the pipeline above it does not care.

Where it meets the rest

digisearch is the hub's research arm alongside digiquant: where the quant vertical answers from market data, search answers from documents. The chat surface leans on it for grounded answers, and the research pipelines call it for background. Ingestion can be scheduled or on demand; either way it lands in the same index the answering path reads.

GO DEEPER · ARCHITECTURE

digisearch architecture →

The two paths, the index choices and the access rules.

Running it in each home

The pipeline runs in every home; the index backend is what changes. Locally an embedded index keeps the whole loop on one machine. On Cloudflare the index lives in managed vector storage beside the workers. Self-hosted, the operator picks the backend that fits the corpus size. Documents cross no boundary the operator did not approve in any home — ingestion is always an explicit act.

Trade-offs

Separating ingestion from answering doubles the machinery for a reason: mixing them is how systems end up with answers that quietly rewrite their own sources. The cost is operational — an index to maintain, ingestion to schedule, stale passages to expire. Chunking is the permanent judgement call: passages too large dilute meaning, too small lose context, and no setting is right for every corpus.

Roadmap

Search improves along retrieval quality: better ranking, fresher indexes, and clearer provenance on every passage an answer cites. The two-path shape holds — it is what keeps the evidence honest.

> **Why digithings.** Grounded answers usually mean sending your documents to a hosted search vendor. Here ingestion, indexing and retrieval all run on infrastructure you control, over standard protocols, so private knowledge stays private while answers stay checkable.

17.1 Passages, not pages

Retrieval is only as good as what it retrieves, and models read passages, not documents. Ingestion therefore splits each document into passages sized for

reasoning: large enough to keep context, small enough to stay on point. That splitting — chunking — is the quiet craft inside the search vertical.

Three judgements shape it. Boundaries follow meaning — paragraphs and sections rather than fixed character counts — so a passage says one thing. Each passage keeps enough surrounding context to stand alone, because the model meets it without the document around it. And overlap between neighbours is deliberate: a small shared margin means nothing important falls exactly on a cut. None of these has a universally right value; each corpus tunes them by reading retrieval quality, not by theory.

GO DEEPER · ARCHITECTURE

digisearch architecture →

Ingestion settings and how to tune them per corpus.

> **Why digithings.** The chunking, the index and the tuning all run openly on your infrastructure. When retrieval disappoints, you inspect the passages and adjust — rather than filing a ticket about someone else's black box.

18

The Cloudflare stack and its workers

The same system at the edge: workers, containers and bindings. What moves to the edge unchanged, and what the edge changes.

Cloudflare is one of the three homes, and the only one where the system runs on someone else's computers without becoming someone else's product. Workers run the request paths at the edge, containers carry the workloads that need a full machine, and bindings wire them to state. This chapter is about what that home looks like from the inside — and what it deliberately does not change.

What it is for

The audience question is portability: if the system runs on a laptop today, what does it take to run it at the edge tomorrow? The answer the stack gives is: a deployment decision, not a rewrite. The same services, the same contracts, the same identity checks — placed on Cloudflare's network instead of a local machine. An operator outgrows a laptop without outgrowing the system.

How it works

Three Cloudflare shapes cover the stack. Workers serve the request-driven paths: the chat interface, the request layer, scheduled runners. Containers host the workloads that need persistent processes — model runners, research pipelines, tracing backends. Bindings connect the two to durable state and to each other, so a worker can reach a containerised service as if it were local. Identity and scopes work exactly as they do locally, because the services verify credentials themselves rather than trusting the network around them.

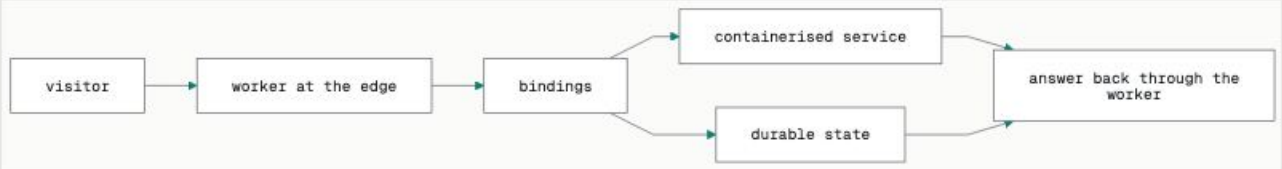


Figure 1

Nothing in this picture is Cloudflare-specific in design — only in address. The services do not know which home they are in, which is why the same build can move between homes without code changes.

Where it meets the rest

The edge home meets the operator through deployment, not through new concepts. Configuration declares which service lands where; the deploy pipeline builds and ships; scheduled runners keep their timetables. The local and self-hosted homes stay available for development and for operators who want every byte on their own hardware. Moving between homes is a supported direction in all directions, not a migration with no return.

GO DEEPER · REFERENCE

Deployment reference →

How each service is placed, built and shipped in every home.

Trade-offs

The edge home trades control for reach. Global presence, no servers to feed, and billing that follows traffic — against the reality that someone else's computers hold the running processes. The architecture limits what that concession costs: stateless request paths, credentials that expire in minutes, and state in stores the operator chose. An edge deployment that assumed the network was friendly would be a different, weaker system.

Roadmap

Edge work follows the platform: newer runtime shapes adopted as they mature, placement declared rather than hand-built, and the local home kept as the development default so the edge never becomes the only place the system can be understood. The invariant is symmetry — no feature that only exists at the edge.

> **Why digithings.** Edge deployment here does not mean re-platforming onto a vendor's framework. The system deploys to the edge the way it deploys anywhere: the same services, the same open protocols, usage-based billing with no hosted-tier features to envy. Reach without lock-in.

18.1 Placing each workload

Placement starts from a simple question: does this workload answer requests or keep processes? Request-shaped work — interfaces, request layers, scheduled runners — becomes workers that scale to zero and wake per request. Process-shaped work — model runners, pipelines, tracing backends — becomes containers that stay up and hold state in memory. The rule of thumb holds surprisingly far: stateless request handling at the edge, stateful processes in containers, durable facts in managed stores.

Scheduled work deserves a note. Runners wake on timetables, do their job and stop — the cheapest shape on the platform, because idle time bills nothing. Long research runs go the other way: start a container, let it work, shut it down. Matching the workload to the shape is most of the cost discipline; the billing follows the placement rather than needing separate attention.

GO DEEPER · REFERENCE

Deployment reference →

The placement table and how to change it.

> **Why digithings.** Placement is declared and reviewable, not clicked together in a console. The same declaration an operator reads is what the pipeline ships

> — so the edge deployment stays as auditable as the laptop one.

19

The local stack, one machine

The whole system on a laptop, composed plainly. Why the local home is the default for development and a legitimate home for production.

Before the system runs anywhere else, it runs here: one machine, every service beside its neighbours, the full loop from question to answer on hardware the operator can touch. The local stack is not a demo mode or a crippled trial. It is the same system, composed for one machine, and for many operators it is the only home they ever need.

What it is for

The audience question is sufficiency: how much machine does this actually need? The local stack answers by being ordinary. Services start side by side, talk to each other over the machine's own network, and keep their data in local stores. A developer runs the same composition they ship; a small firm runs the whole product on one well-chosen box. Nobody needs permission, an account, or a card on file to begin.

How it works

Composition is declarative: one description says which services run, what each needs, and how they reach each other. Starting the stack brings them up in dependency order; stopping it puts everything away. The services behave identically to their siblings in other homes — same contracts, same identity checks, same tracing — because the code never learns which home it is in. The only local concession is convenience: friendlier defaults, warmer caches, and tooling that assumes a person is watching.

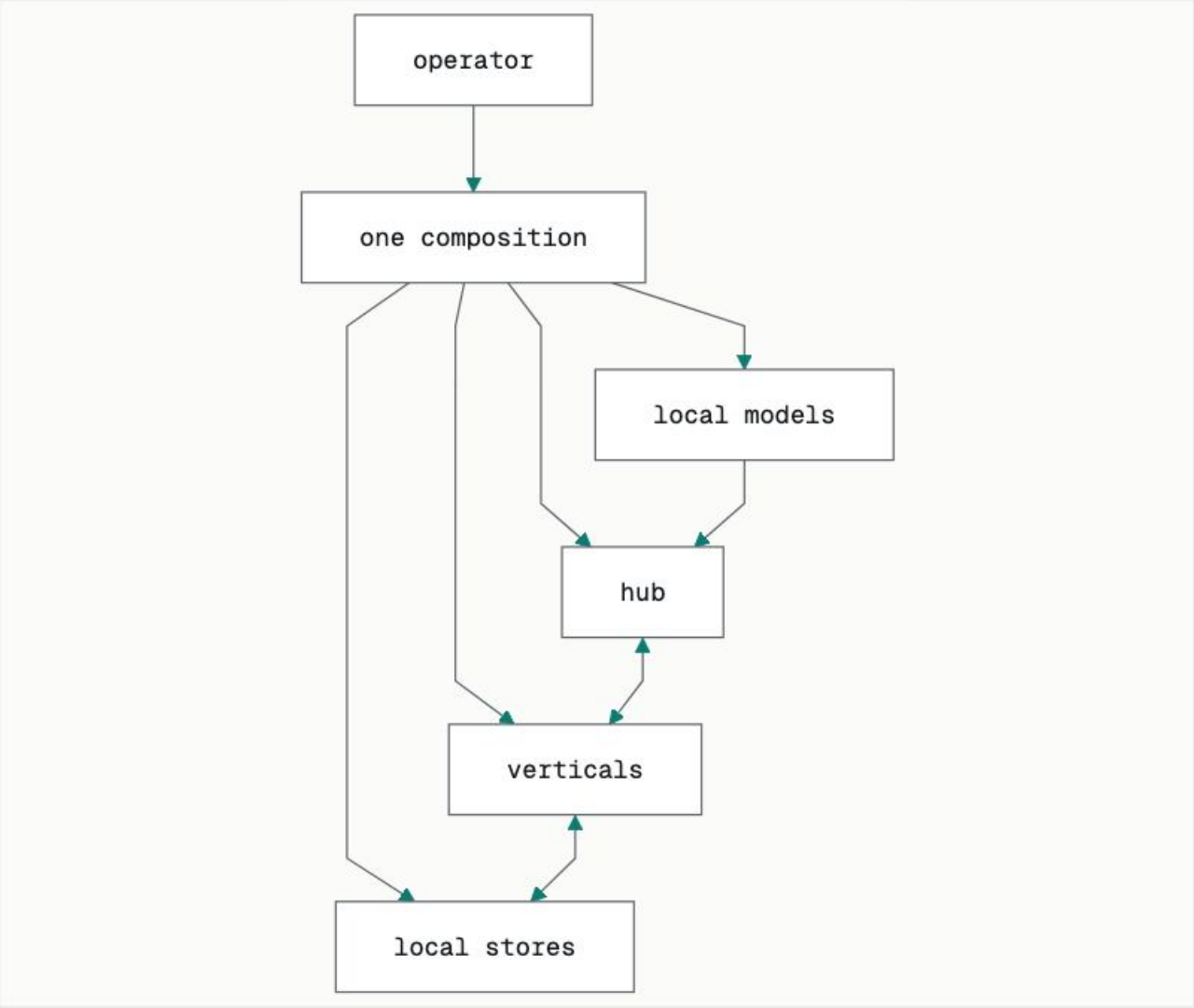


Figure 1

Seed content and example flows ship with the composition so the first run answers real questions immediately. From there the operator grows into the system by changing configuration, never by reinstalling it.

Where it meets the rest

The local home meets the operator's working day: development, experiment,

demonstration, and honest small-scale production. When traffic or appetite outgrows one machine, the same services move to the edge or to self-hosted hardware with their contracts intact. Nothing written against the local stack needs rewriting — that guarantee is what makes the local home safe to invest in.

GO DEEPER · GUIDE

Local stack guide →

Starting, stopping and understanding the one-machine composition.

Trade-offs

One machine is finite: memory, accelerators and disk set real ceilings, and a laptop that sleeps pauses the system with it. Backups are the operator's job, not the platform's. Against that, the local home offers zero marginal cost per question, total data custody, and the fastest possible development loop. The system is frank about the ceilings — scaling guidance names them — so outgrowing the home is a planned step, not a surprise.

Roadmap

Local tooling improves toward less setup and clearer ceilings: faster first starts, plainer status, and guidance that says exactly when one machine stops being enough. The composition stays the reference the other homes are measured against — if it works locally, the design is sound.

> **Why digithings.** A serious AI stack that starts on a laptop with no account and no meter is rare enough to be the point. The local home means evaluation costs nothing, development stays fast, and small operators never pay for scale they do not have.

19.1 The first run

The first run should feel uneventful. Fetch the checkout, start the composition, and watch each service report healthy in turn. Seed content loads on its own, so the chat answers questions and the examples resolve immediately. Total time is measured in minutes, most of it spent fetching model files on first use.

When something does not start, the order of inspection is fixed. First the composition's own status: which service is unhealthy, and what did it log last? Then dependencies: did storage and the model runner come up before the services that need them? Then resources: is there memory and disk to spare? Nine first runs in ten end at one of those three steps. The tenth — a genuinely new

failure — is worth reporting, because it improves the composition for everyone after.

GO DEEPER · GUIDE

Local stack guide →

The commands and the troubleshooting checklist.

> **Why digithings.** Starting serious infrastructure with one command and no account is the evaluation the product deserves. Nothing to procure, nothing to expense, nothing to cancel if you walk away — the first run costs minutes, not a purchasing process.

20

MCP, every capability as a tool

One open protocol carries every tool the system offers and consumes. Why tools are described, discovered and gated — never hard-coded.

A system with dozens of capabilities needs a way to say what each one does that both people and models can read. The industry answer is the Model Context Protocol: tools described with names, schemas and permissions, over a standard transport. digithings adopts it everywhere — every capability the system offers is a described tool, and every model-facing call goes through the same gate.

What it is for

The audience question is extension: how does something new plug in? The answer the protocol gives is uniformity. A new search source, a new analysis, a new integration — each arrives as a tool description, and every consumer in the system already knows how to read descriptions. Nobody patches the hub, the chat or the dashboard to adopt it. Describing is integrating.

How it works

Tools live behind servers that advertise them. The hub and the verticals list what a server offers, filter that list through the operator's allow-list, and expose only what survives to the model. Invocation carries the same identity the rest of the system uses, so a tool call is authorised with the same scopes as any request. A per-server gate decides which tools may run unattended and which need a person to approve — and that gate is checked where the tool executes, not only where it is offered, because offering and executing are different promises.

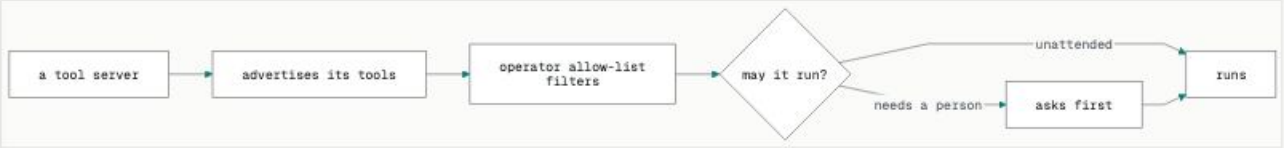


Figure 1

The subtle rule: a model is never offered a tool it could not invoke, and invocation never trusts the offer alone. Both halves are checked, every time, which is what keeps a clever prompt from reaching past its grant.

Where it meets the rest

The protocol is the system's connective tissue. Verticals publish their tools through it; the hub consumes them through it; outside agents and editors connect to the system's own tool servers through it. Operator-owned integrations — project trackers, team chat, ticketing — arrive as tool servers under the same gate, scoped as narrowly as the operator allows.

GO DEEPER · ARCHITECTURE

digigraph architecture →

The hub side: discovery, the per-server gate and the execution check.

Running it in each home

The protocol is transport-agnostic, so homes change nothing about the contract. Locally, tool servers run as neighbours on the machine. On Cloudflare they sit behind the same bindings as everything else. Self-hosted they run wherever the operator keeps services. What changes per home is only which tool servers the operator connects — the gate, the scopes and the approval flow travel unchanged.

Trade-offs

A uniform tool layer centralises risk in one gate: if the gate is wrong, every tool is wrong. The system answers with defence in depth — allow-lists at offer time, the approval gate at execution time, scopes on the identity throughout — so no single check carries the whole promise. The price is configuration the operator must actually read: a tool layer that asks for approval on everything trains people to approve everything. Defaults lean toward asking, and operators narrow from there.

Roadmap

The tool surface grows outward: more operator integrations, finer approval policies, and richer descriptions as the protocol itself matures. The gate stays where it is — at execution — no matter how many servers advertise.

> **Why digithings.** Tool access is usually bespoke glue per integration. Standardising on the open protocol means integrations compose instead of accumulating, and the approval gate means a model with many tools is still governed. Modularity you can audit, not just admire.

20.1 The approval moment

Some tools run unattended; some wait for a person. The approval moment is the interface between the two: the system pauses, shows what it wants to do and with what, and continues only on a yes. Designing that pause well matters more than it looks — an approval a person grants without reading is no approval at all.

Three habits keep it meaningful. Approvals name the action in plain words, not tool identifiers — what will happen, to what, and what it costs. They arrive with the context that justifies them, so the person decides from evidence rather than from trust. And routine approvals can be pre-authorised by policy while novel ones always ask, so attention goes where novelty is. The gate records every decision either way, approved or declined, because the record of what was allowed is as much a control as the asking.

GO DEEPER · ARCHITECTURE

digigraph architecture →

The approval gate and its policies.

> **Why digithings.** A model with dozens of tools and a real approval gate is governed automation, not a demo. Open protocol, operator allow-lists and recorded decisions — oversight that survives contact with production.

21

Data, remembered and retrievable

Structured rows and stored objects, side by side. How the system keeps history it can query and files it can fetch.

Everything the system learns has to land somewhere. Conversation history, research briefs, scheduled digests, market history, uploaded documents — each with a different shape and a different query pattern. The data chapter is about the two stores that hold it all and the discipline that keeps them honest: structured facts in tables, bulk bytes in object storage, and a clear rule for which lives where.

What it is for

The audience question is durability: where does yesterday's work sleep, and will it be there tomorrow? Operators need conversation history that survives restarts, research outputs that accumulate instead of evaporating, and market data cached close to the pipeline that reads it. Two stores answer those needs without pretending to be each other.

How it works

Structured facts — conversations, briefs, digests, schedules, ledger rows — live in relational tables with explicit schemas and migrations. Each change to the shape is a versioned migration, applied in order, so the schema's history is as reviewable as the data. Bulk bytes — market archives, document stores, pipeline checkpoints, backups — live in object storage, addressed by name and fetched whole. The rule for which lives where is about access pattern: queried and joined goes in tables, written once and read whole goes in objects.

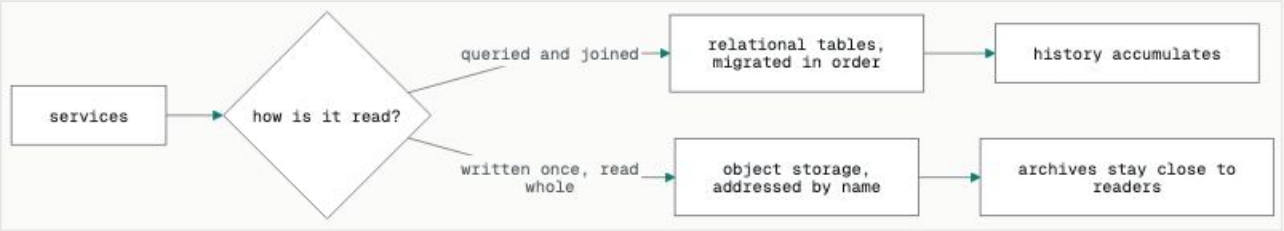


Figure 1

Migrations run automatically with deployment, in the same pipeline that ships the code expecting them — schema and code never drift apart silently. Cached market data refreshes on schedule with provenance attached, so a consumer can tell how fresh the bytes are and where they came from.

Where it meets the rest

Every vertical reads here. The chat surface persists conversations, the research pipelines store briefs and digests, the quant vertical leans on cached market history, and search indexes documents whose originals rest in object storage. Retention is per-kind rather than per-store: conversation history, research outputs and caches each age out on their own policy.

GO DEEPER · REFERENCE

Deployment reference →

Migrations, stores and how each home wires them.

Running it in each home

The shapes stay constant; the backends vary by home. Locally the tables live in a nearby database and objects on local disk — fast, free, and the operator's to back up. On Cloudflare the tables move to the platform's database offering and objects to its object storage, reached through bindings. Self-hosted, the operator picks equivalents and points the services at them. In every home the migration order and the access rules are identical; only addresses change.

Trade-offs

Two stores means two things to operate: schemas to migrate and buckets to retain. The alternative — one store pretending to do both jobs — has been tried everywhere and bills its confusion forever. The honest costs are migration discipline (every shape change reviewed, ordered and reversible where possible) and retention judgement (history is valuable until it is a liability, and the line moves per kind).

Roadmap

Data work heads toward clearer provenance and calmer retention: every row and object able to say where it came from and how long it stays. The two-store shape holds — it matches how the system actually reads.

> **Why digithings.** Your history and your archives live in standard stores you can inspect with ordinary tools, on infrastructure you chose. No proprietary data layer standing between you and everything the system ever learned for you. Ownership that includes the past.

21.1 Migrations in order

A database schema is a promise the code relies on, and promises change. Migrations are how the system changes them safely: each shape change is a small versioned step, applied in order, each one moving the schema from one known state to the next. The pipeline applies pending steps automatically on deploy, so the schema the code meets is always the schema the code expects.

The discipline is in the ordering. Every migration names its predecessor, steps never branch, and a step that already ran never runs again. Review treats migrations like any other change — readable, reversible where possible, and tested against a copy before touching anything real. When a deploy and its migrations travel together, schema drift stops being a category of incident and becomes a routine: describe the change, review it, ship it, move on.

GO DEEPER · REFERENCE

Deployment reference ⟶

Migration automation per home.

> **Why digithings.** Schema history in versioned steps, applied by the same pipeline that ships the code, on databases you operate. Data stewardship as ordinary engineering — reviewable, repeatable, and yours.

22

Running the stack, safely

Operating digithings means holding three views at once: what the system is doing, who is allowed to do what, and where the proof went. Why those views are separate, and what each one is for.

A system that researches, searches and analyses on your behalf is a system you must be able to watch. Operating digithings is not a separate discipline bolted onto the side — it is three views of the same running system, each answering a question the others cannot. What is happening right now? Who is allowed to make it happen? And where is the record, so yesterday's run can be replayed and understood?

The audience question this chapter answers is the operator's: can I trust this stack to run unattended, and when something surprises me, where do I look first? Answer: in three places, each with one job.

What it is for

The first view is observability: traces of every request, health signals from every service, and an audit trail of consequential actions. The second is security: identity on every hop, least privilege on every credential, and a default posture that assumes a hostile network and a single trusted machine. The third is the boring machinery that makes the other two possible: reproducible deploys, smoke checks after every release, and one operator runbook per surface.

None of the three substitutes for the others. A green health signal does not prove a request was authorised; a valid credential does not prove the system behaved; and a passing test suite does not prove the deployed build is the one that was tested. The chapters below keep the three views apart on purpose, so each can be checked independently.

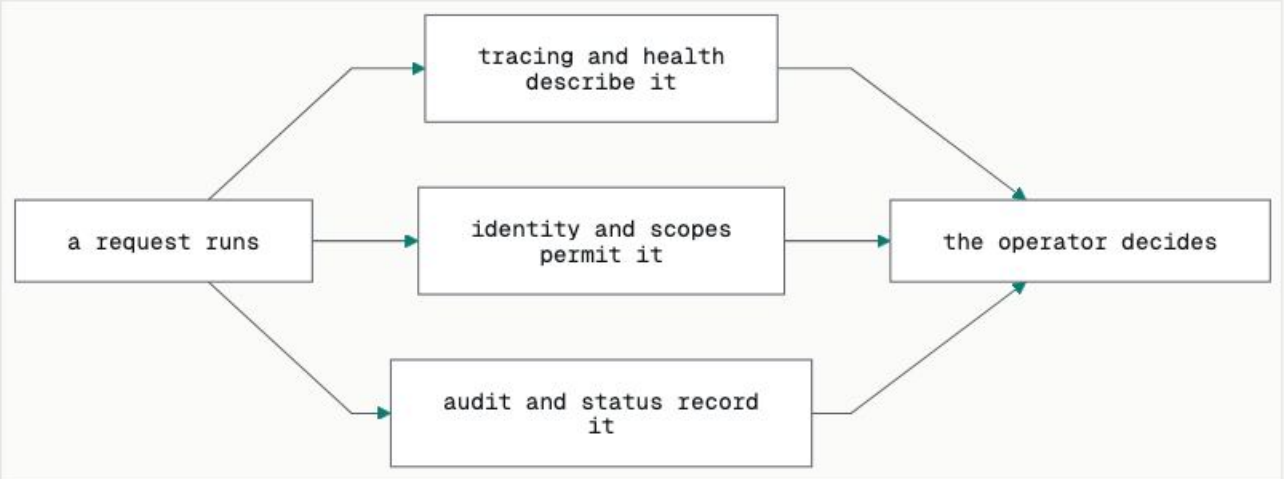


Figure 1

Where it meets the rest

Operations touches every module and owns none of them. The hub reports which tools ran; the model layer reports what each call cost; the data layer reports what was stored and when. Identity arrives from the one issuer and is checked at every edge, so the operator configures trust in one place and every service enforces it. The tracing library is deliberately backend-neutral: spans flow to whichever backend the operator chose, and the book never names one as the answer.

GO DEEPER · ARCHITECTURE

Security posture →

The threat model, the non-negotiable defaults, and how to report a problem.

Running it in each home

The three views travel with the stack. On a laptop the same health signals answer, the same audit trail appends, and the same identity checks run — only the backends are smaller and local. On Cloudflare the signals come from the platform's own telemetry alongside the stack's, and the audit trail is the deploy's memory of itself. Self-hosted, the operator picks the backends and keeps the same three questions. Portability of the views is what lets a run verified locally predict behaviour in production.

Trade-offs

Three views cost three kinds of attention. On the good side, a surprise is cheap to place: a slow answer is a tracing question, a refused call is an identity question, and a disputed action is an audit question — and the wrong view fails loudly rather than answering for another. On the sharp side, no single dashboard tells the whole story, and the audit trail is only as honest as the process that writes it. The stack accepts that cost because the alternative — one blended signal — fails silently in exactly the cases that matter.

Roadmap

The direction is remote sinks for the audit trail, per-tenant quotas beside the per-address limits, and a runtime interlock where today there is a source-tree gate. None of that changes the three questions — only how completely each one is answered.

> **Why digithings.** Most stacks treat operations as the price of admission: closed dashboards, proprietary agents, telemetry that only flows one way. digithings keeps the views open and standard — open protocols for discovery and tracing, reproducible deploys from the same source anyone can read — so watching the system never depends on renting the watcher.

22.1 Seeing what runs

A request that cannot be replayed cannot be understood. Every request the hub routes carries identifiers from edge to edge, and each step it takes — which tools ran, in what order, how long each took — is recorded as a span beside the answer. When an answer surprises you, the trace is the first thing you read: it shows the path the request actually took, not the path you expected.

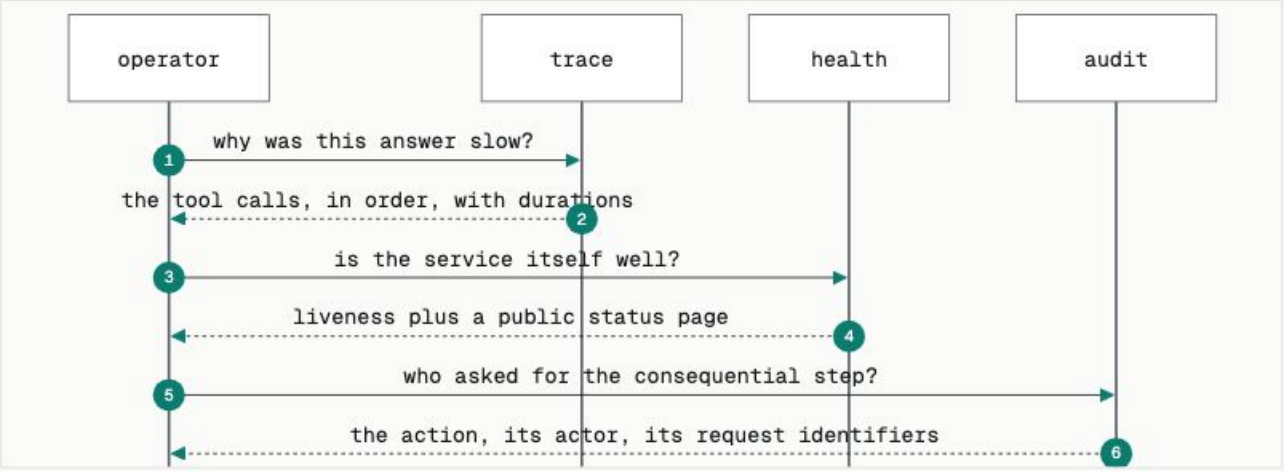


Figure 1

Three properties keep the instruments honest. The trace identifiers travel with the request, so a span from any service joins the same story. Health signals are deliberately minimal and free of secrets, so they can be probed continuously and even publicly without leaking anything. And the audit trail records consequential actions immutably, with the identifiers that tie each entry back to the trace that caused it.

GO DEEPER · ARCHITECTURE

Security posture →

Which signals are public by design, and which records stay private.

> **Why digithings.** A surprise you can replay is a question; a surprise you cannot is an incident. Open tracing over standard protocols means the replay never depends on a vendor's dashboard being available.

23

Your machine first

The stack runs on your own machine as a first-class variant, not a development hack. Why local-first is the pre-production gate, and how one shared contract keeps it honest.

Most infrastructure projects treat running locally as a favour to developers: a smaller, vaguer cousin of the real thing. digithings treats your machine as a first-class home. The stack runs there fully — hub, verticals, chat, tracing — and anything verified locally is expected to behave the same way in production, because both homes read the same contract.

The audience question this chapter answers is the sceptic's: why should I believe a demo on a laptop predicts production at all? Answer: because the two variants are not allowed to drift apart, and the checks that prove it run on every change.

What it is for

Self-hosting has two jobs. The first is independence: the software works without any hosted platform, so owning your infrastructure is a real option, not a slogan. The second is verification: interfaces, chat surfaces and tool integrations are proven fully connected locally before anything ships to the hosted platform. Local-first is the gate production walks through, not the waiting room beside it.

That only works if the two variants share one contract: the same setting names, the same service layout, the same secrets manifest. A variant supplies values; it never invents a name. Parity checks compare the two homes continuously, and drift fails loudly instead of accumulating quietly.

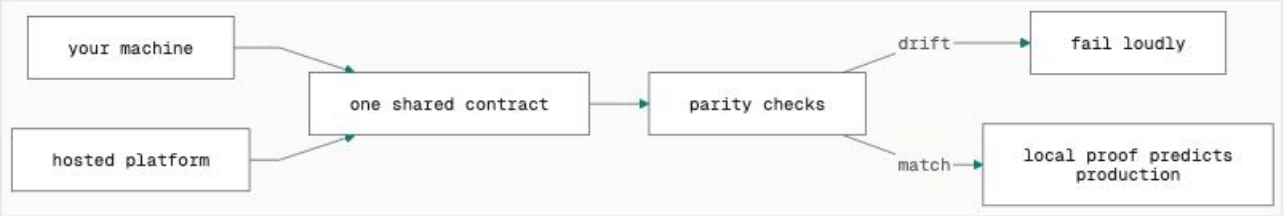


Figure 1

Modularity follows the same principle. Each capability opts in separately, so you run only what you need — the core set, the chat surface, the tracing backend, the tool integrations — and each addition is one explicit choice rather than a bundle you accept whole.

GO DEEPER · GUIDE

Self-hosting reference →

The full operator document: prerequisites, profiles, and the plan slice by slice.

Where it meets the rest

The self-host variant touches everything the hosted one does: the same hub routes, the same model layer answers, the same data layer persists. The local model runner stands in for hosted inference; the local database stands in for the hosted one. Nothing in any module prefers one home — that neutrality is what the parity checks are actually testing.

Running it in each home

This chapter describes the home you own. The hosted platform is production only: there is no hosted development stack, because duplicating the whole infrastructure for development would double its cost. Development, review and verification happen on machines; production is where verified work lands. The asymmetry is deliberate and one-directional.

Trade-offs

A local-first gate is a bet with two sides. On the good side, every developer holds the whole system, surprises surface before they cost money, and independence is demonstrated continuously rather than claimed annually. On the sharp side, one machine is not the hosted platform: load, scale and multi-tenancy are verified by reasoning and by narrower checks, not by reproducing production locally. The book says that plainly so no reader mistakes the gate for a guarantee.

Roadmap

The plan completes the variant slice by slice: the config contract, local data services, worker emulation, one command to raise the stack, seeding, and the parity checks that hold it all together. Each slice lands as reference documentation first, so the target state stays readable while it is still being built.

> **Why digithings.** Most platforms let you rent their cloud or leave. digithings proves, on every change, that its software runs without them — open source, reproducible steps, the same contract in both homes. That is the self-hosting position in one practice: independence you can verify.

23.1 One contract, two homes

Two homes for one stack only stay honest through a shared contract. The contract names every setting, every service address shape, and every secret slot once. Your machine and the hosted platform each supply values for those names. Neither side renames, reorders or reinterprets them — because the moment one side invents a name, local verification stops predicting production behaviour.

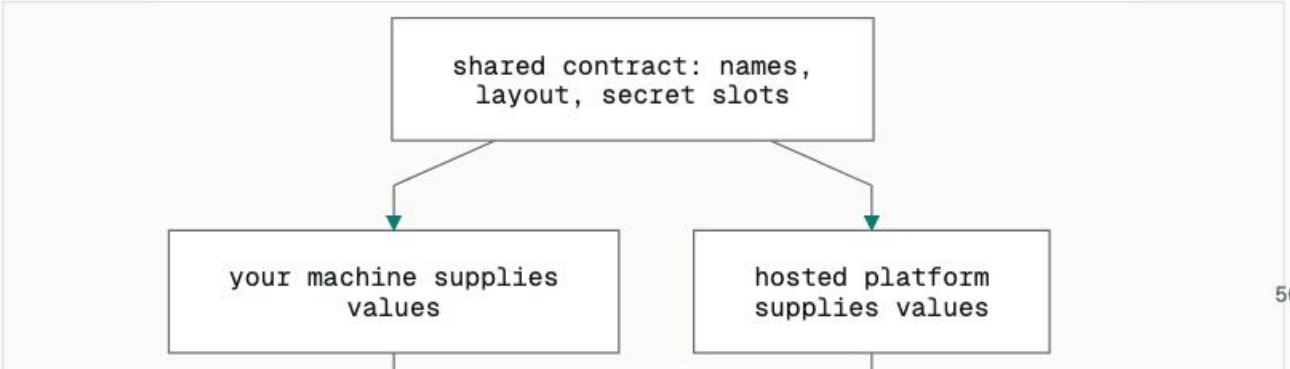


Figure 1

Three properties make the contract hold. Settings are named once, in one manifest, and both homes read from it. Secrets are rendered per home from that manifest with strict file permissions, and never echoed. And the parity checks run automatically: they compare the two homes' settings and fail the change when they disagree, so drift is caught at review time rather than discovered during an outage.

GO DEEPER · GUIDE

Self-hosting reference →

The contract, the profiles, and what each plan slice adds.

> **Why digithings.** Configuration drift is how most self-hosted setups die: the documented path rots while production moves on. A contract both homes read, checked on every change, keeps the documented path alive.

24

How a change lands

Small focused changes, real tests, review by people, and a signed record of origin. What the project asks of contributors, and why each ask exists.

An open-core project lives or dies by how it treats changes from people it has never met. Too little process and the stack rots; too much and nobody contributes twice. digithings asks for a small set of things, each with a reason an outsider can check: work scoped to an issue, changes small enough to review, tests that actually exercise the claimed behaviour, and a signed statement that the contributor has the right to submit the work.

The audience question this chapter answers is the newcomer's: what do I have to do for my change to be taken seriously? Answer: less ceremony than you fear, and more honesty than you may expect.

What it is for

Every contribution rule defends something concrete. New capabilities arrive as discoverable tools, so the hub can route to them without special cases. Quantitative code goes through the one backtesting core, so results stay comparable. Models are reached through the one router with caching, so token spend stays visible. Components ship in containers from the root layout, so anyone can reproduce the running system. And interfaces document their own changes where they live, so the architecture notes never lag the code by more than one change.

None of this is taste. Each rule exists because violating it once cost the project real time: a special-cased capability the hub could not see, two backtest numbers nobody could compare, a model bill nobody could explain. The rules are the project's memory of those incidents.

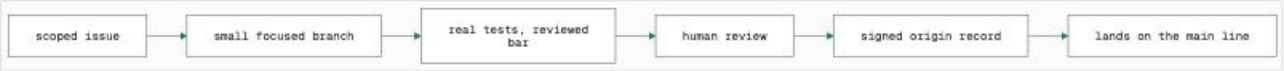


Figure 1

Where it meets the rest

Contributing touches the whole repo but lands through one path: an issue for scope, a branch for the work, a pull request with the template filled in — what changed, why, how it was tested. Automated checks run first: style, unit tests, documentation links. People review second, and some changes always need a person: authentication, live-trading paths, new infrastructure, or any decision the records do not cover yet. Those go through an architecture record before they go through review.

GO DEEPER · GUIDE

Contributing guide →

The required reading, the hard rules, and the workflow step by step.

Running it in each home

Contributing is the same in every home because it happens before any home matters: the change is written, tested and reviewed against the shared source, and only deployed homes differ afterwards. A contributor never needs production access; the local variant from the previous chapter is the whole verification

environment.

Trade-offs

A review bar is a bet with two sides. On the good side, every change carries its reasoning and its proof, so the project stays legible as it grows and a newcomer can trust the history. On the sharp side, review takes human time, and the bar can feel slow to someone with a one-line fix. The project accepts that cost because the alternative — fast merges nobody understands — compounds until nobody can change anything safely.

Roadmap

The direction is faster honest feedback: more checks that run before a person looks, clearer ownership per surface, and records for the decisions that currently live only in discussion. None of that lowers the bar — it moves the waiting earlier, where it costs less.

> **Why digithings.** Many open projects optimise for stars: easy to start, hard to trust. digithings optimises for the opposite — a contribution path strict enough that running the result is safe, and open enough that anyone can walk it. That is the open-core position in one practice.

24.1 The shape of a good change

A good change has four parts, and a reviewer checks them in order. First, scope: the work answers one issue, so its purpose is written down before the code exists. Second, size: the branch stays small and focused, so a person can hold the whole diff in mind at once. Third, proof: tests exercise the claimed behaviour for real — no trivial assertions, no stubs standing in for the path being claimed, and the end-to-end journey still passing. Fourth, origin: every commit carries a signed statement that its author has the right to submit it, so the project's history is also its licence record.

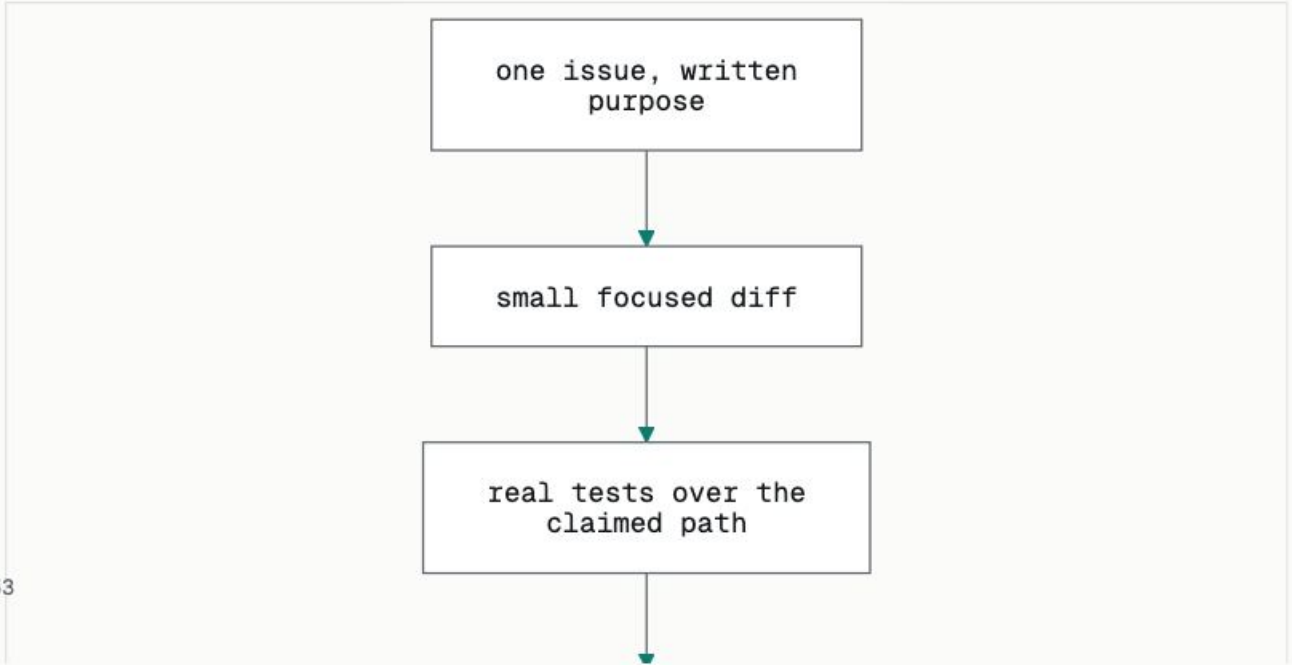


Figure 1

Two properties keep the bar honest rather than theatrical. The automated checks run before any person looks, so mechanical problems never spend human time. And the origin statement is checked mechanically too — a change without it cannot land no matter how good it is — because a licence record that depends on reviewers remembering to ask is not a record at all.

GO DEEPER · GUIDE

Contributing guide →

Titles, tests, review policy, and the origin statement in full.

> **Why digithings.** Trust in open source is a property of the history, not of the people. A change a stranger can verify — scope, proof, origin — is a change anyone can build on.

25

Why these parts

A router for models, workers at the edge, one database family, open tool protocols, and charts from a shared library. Each choice against its alternatives, with the record of where the map was once wrong.

Every stack is a pile of choices its authors stopped questioning. This chapter re-opens them: for each major integration — the model router, the edge platform, the data layer, the tool protocols, the charts — what it does, what else could have done it, and why this one won. The point is not to flatter the choices. It is to leave the reasoning where the next author can argue with it.

The audience question this chapter answers is the architect's: if I had to replace a part, which parts are load-bearing and which are convenience? Answer: the protocols are load-bearing; the vendors are not.

What it is for

Models are reached through one routing layer with caching and spend visibility, instead of every service holding its own provider accounts. That single decision is what keeps model spend explainable and what lets a local runner stand in for hosted inference without touching any caller. The alternative — each service integrating providers directly — was rejected because its bills cannot be reconciled and its failures cannot be compared.

Compute at the edge runs on the workers platform: the chat surface, the data API and the scheduled jobs all run there, with durable state and object storage from the same provider. The alternative — serving everything from containers on rented machines — was rejected for the surfaces where global placement and zero upkeep matter, and kept where they do not: the chat container still runs beside its worker, because some workloads want a full machine.

Data lives in one database family with migrations as the contract, an archive object store beside it, and a vector backend behind the search vertical. One family means one backup story, one migration story, and one place where the retention rules live. Tools are described over the open tool protocol and discovered at runtime, so a new capability joins by describing itself rather than by patching the hub — the modular position from the architecture chapter, applied to vendors.

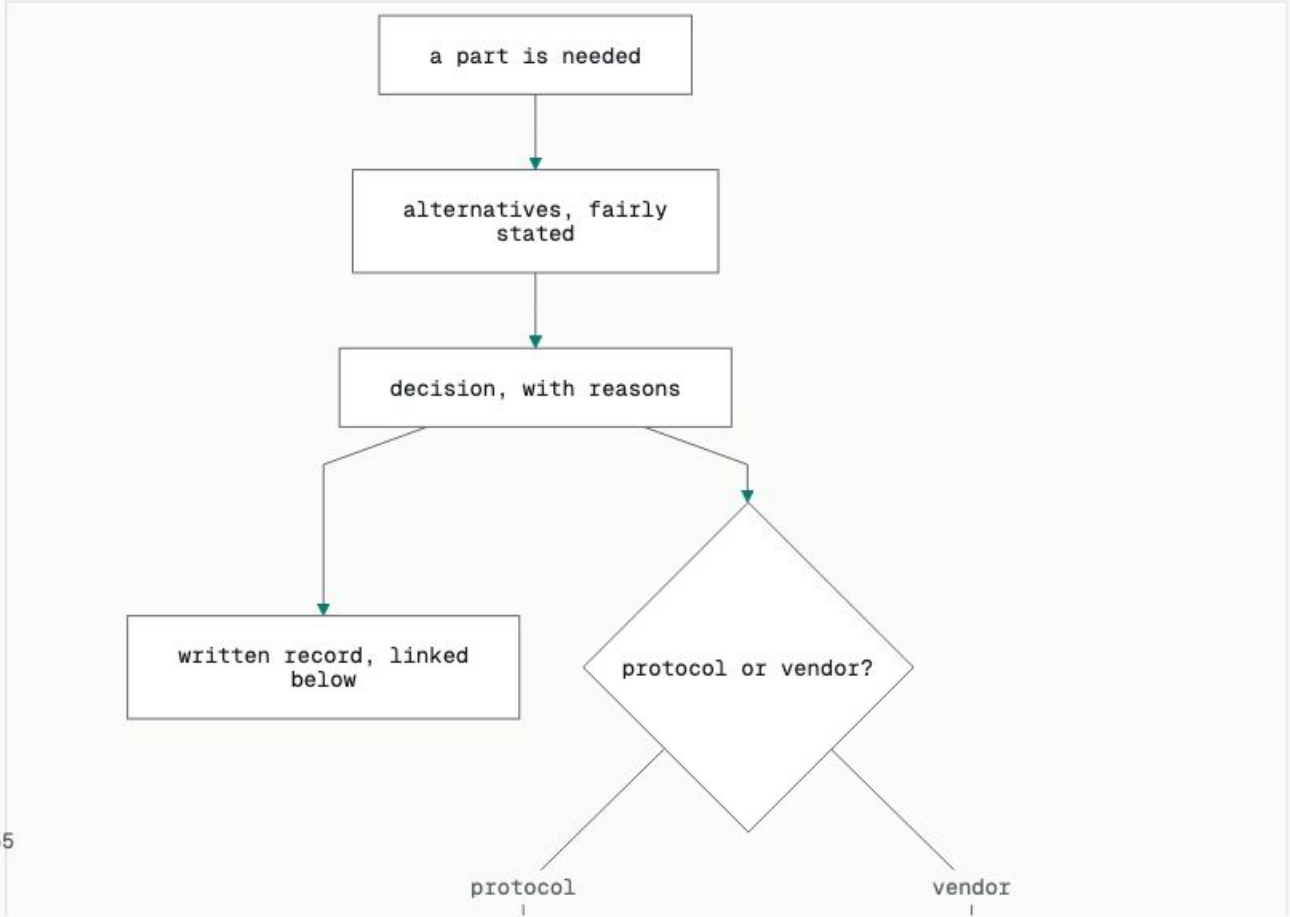


Figure 1

Charts come from a shared visualisation library rather than bespoke rendering per surface, so every dashboard, report and embedded view speaks the same visual language. The alternative — each surface drawing its own — was rejected because inconsistent pictures of the same numbers erode trust faster than any single bug.

GO DEEPER · GUIDE

Architecture decision records →

The written record of each decision, newest first, each immutable.

Where it meets the rest

Integration choices constrain every chapter before this one and are constrained by the two after it. The hub chapter assumes runtime tool discovery; the chat chapter assumes the edge worker; the data chapter assumes the migration contract; the operations chapter assumes the tracing backend. Changing any choice in this chapter ripples outward — which is exactly why each choice links its written record instead of resting on this chapter's prose.

Running it in each home

Choices are evaluated per home, not once globally. The model router runs in both homes with different providers behind it. The edge platform is production only; locally, ordinary processes stand in. The database family is the same in both homes at different sizes. A choice that only works in one home is a liability the contract chapter calls out, not a detail to discover during an outage.

Trade-offs

Documented choices are a bet with two sides. On the good side, the next author inherits reasoning instead of archaeology, and replacing a vendor behind a stable protocol is ordinary work. On the sharp side, records go stale — and the section below is the honest example this book caught itself. A decision record that nobody re-checks becomes folklore with a number on it. The review loop from the findings appendix exists to stop that: writing surfaced the drift, the drift became a ticket, and the record below is what the ticket established.

Roadmap

The direction is fewer vendors behind steadier protocols: more of the system

described over open standards, fewer bespoke integrations, and records revisited on a rhythm rather than by accident. Each new record follows the same template, so the collection stays arguable-with rather than archival.

> **Why digithings.** Most vendors sell the bundle: their models, their cloud, their database, their charts — each convenient, all of them together a lock-in. digithings picks each part against alternatives, writes down why, and keeps the protocols open so any part can leave. That is the no-lock-in position in one chapter.

25.1 When the map was wrong

Writing this book caught its own source disagreeing with the system. The deployment reference described the chat's edge scaffold — worker plus container — as removed on a past date, with no deploy path, on the grounds that containers need a paid platform tier the project did not hold. Both halves of that sentence had gone stale: the scaffold is live in the tree, it serves real traffic on the main site, and the platform tier it needs is the one the project holds.



Figure 1

The corrected record, which this section keeps: the chat edge scaffold is present and live, it requires the paid tier, and the chat surface ships through two paths — the worker and container on the edge, and the chat service on operator machines for the project's own use. Those are different surfaces with different deploy paths, and the reference now names both instead of describing one as gone.

Two properties make this section load-bearing rather than anecdotal. First, it is the worked example of the review loop: writing surfaced the drift, the drift became a ticket under the book's epic, the ticket went to the owner of the deployment surface, and the reference was corrected there — not in the chapter that found it. Second, it records the failure mode honestly: a sentence that was true when written, stale a season later, with no check to catch it. The parity thinking from the self-hosting chapter applies to documents too — a claim nobody re-checks is folklore, however it reads.

GO DEEPER · GUIDE

Deployment reference →

The corrected operator document, naming both chat surfaces and

```
their lanes.
```

> **Why digithings.** A book that hides the mistakes it found is marketing. Keeping the corrected record — what was claimed, what the tree said, and where the fix landed — is what lets the next reader trust the rest.

91

Reading and printing

Pan and zoom on screen, paginate on paper, and build the file in CI so the printed copy is a build artifact rather than somebody's browser.

The book is read two ways and both are supported deliberately.

91.1 On screen

Diagrams are ordinary images until you ask for them to be bigger.

Dragging a diagram opens it: the image pans and zooms under the pointer, with the wheel and with a pinch. It is an interaction on top of an image, so the image is still there when the interaction is not — no script, no figure.

91.2 On paper

- running headers carry the chapter title and the book title;
- page numbers sit in the outer margin, which alternates by page parity;
- the table of contents on the printed front matter carries page references, resolved after the document is paginated.

The last point is the one worth explaining. A page number is not knowable until the engine has decided where the pages break, so it is not written into the source. The engine fills it in, after pagination, by looking up the anchor each entry points at. A number written by hand would be wrong the moment a paragraph grew a line.

Why the print step is a build step

The printed book is produced by a headless browser in CI, and the resulting file is uploaded as an artifact. It is not produced by a person pressing print.

That decision costs a browser in the pipeline and buys three things: the file is reproducible from a commit, the same input always yields the same output, and nobody's machine settings — paper size, margins, "fit to page", a stray header and footer — end up in the printed book.

91.3 What the two outputs share

The diagrams, the anchors, the running headers, and the page geometry all come from one page. A change that improves reading improves the print, because there is only one page.

92

What writing surfaced

Writing the book from the design documents surfaced places where the documents and the running system disagree. Each became a ticket, so the book improves the system it describes.

A book written from design documents is also an audit of those documents. Twice now, writing a chapter surfaced places where the written design and the running system say different things. Each one became a ticket under the book's epic, with the chapter section that surfaced it linked, so the review loop is part of how the book gets written rather than a separate activity.

Findings come in two kinds, and they close differently. **Drift** means a document went stale while the system moved on: the fix is to update the document, and anyone can verify the fix by reading the two side by side. **Decision** means the document and the system disagree about intent — whether some work is planned or done, or which shape is meant to win. Editing prose on a decision item produces a document that reads confidently and is wrong, so those wait for a ruling first and only then get edited.

Each finding goes to the seat that owns the surface it describes, even when the stale words sit in a document someone else touches. The owner of the facts fixes the words; the owner of the page reviews them.

The loop also records its misses. Two candidates from the architecture chapter turned out to be false: both were measured against an out-of-date copy of the tree, and the documents were right. A process that only records hits teaches its authors nothing about checking the current tree first, so withdrawals stay in the record with the reason they failed.

The live list of findings lives on the board under the book's epic, where each ticket carries its status. This appendix records the process, the template every finding follows, and what has been found so far.

GO DEEPER · GUIDE

The digithings vision →

The design documents this loop audits, and the first place drift shows.

92.1 The finding template

Every finding ticket follows the same shape, so a reader can tell in the first screen what is claimed and what would settle it.

The title names the mismatch in one line. The body opens with the verdict: what should be true, and what kind the finding is. **Drift** closes when the document matches the tree again. **Decision** closes only after a ruling picks which side wins, and says who gives it. Evidence comes after the verdict, never before it: the board truncates long descriptions in list views, so a template that buries its conclusion reads as a brief that stops mid-sentence.

The ticket links the chapter section that surfaced it, and it is assigned to the seat that owns the described surface. Four of the first five findings were stale words in shared documents, and all four went to the owners of the facts rather than the owners of the pages.

A finding that turns out to be false is withdrawn, not deleted, with the reason it failed. The usual reason is a stale copy of the tree: the author measured

against a checkout that had fallen behind, and the document was right all along. Recording that keeps the next author honest about checking the current tree before filing.

92.2 Findings so far

The architecture chapter produced the first five findings. The deployment reference described the chat scaffold as removed and its plan as free, while the running chat worker serves live traffic. No automated lane ships two of the production stacks; both ship outside the pipeline. The provider guide claimed a weekly routine the scheduler no longer keeps. The vision material calls the model library a future migration, while the hub already relies on it across many modules. And the vision material says the chat is not public, while the live chat serves the main site. Two further candidates were withdrawn after re-measurement against the current tree.

The foundations review, done while collapsing the early chapter tickets into this batch, checked the architecture chapter claim by claim against the running tree. The module map, runtime tool discovery, identity checks on every hop, the routing layer image, static hosting, the archive store, the vault format, the chat running with no database in production, and the self-hosted tracing all hold. No new mismatch was filed.

The review settled two things directly. The chapter order had two chapters sharing one position, so the spine now reads from motive to mechanism: why a book, why self-hosted, how the system fits together, how it scales, and what it costs. And the chapter maps the request path — hub, verticals, identity, models, data, chat — while the voice interface, the skill compiler and the developer tooling stay out of scope until they join that path. That scope is a decision, recorded here, not an oversight to re-file.

COLOPHON

the digithings book

Set in the digithings mono voice. Built from the same files as the web edition at digithings.ai/book.

Edition 2026 · digithings.ai

digithings